

Wiki de Conocimiento y Defensa

Detección (YOLOv11) + OCR CNN + Lógica Difusa Takagi-Sugeno

Material de Estudio para Defensa del Proyecto
Inteligencia Artificial — Universidad Técnica de Ambato

Junio 2026

Índice

1. Arquitectura del Sistema y Decisiones de Diseño	2
1.1. Pipeline de Procesamiento (Diagrama)	2
1.2. El Debate Crítico: OCR Síncrono vs Asíncrono	2
2. Visión por Computador Avanzada	2
2.1. Preprocesamiento y Resoluciones: El Umbral de 48px	2
2.2. Binarización: Umbral Adaptativo Gaussiano vs Otsu	3
2.3. Decodificación Posicional Categórica	3
3. Fundamentos Teóricos de Machine Learning y Deep Learning	3
3.1. Machine Learning Clásico vs Deep Learning	3
3.2. Conceptos Nucleares y Matemáticas del Entrenamiento	3
4. Anatomía Profunda de una CNN (Convolutional Neural Network)	4
4.1. Convoluciones vs Redes Densas	4
4.2. Jerarquía de Características	5
4.3. MaxPooling y Arquitectura ResNet	5
5. Profundización en YOLOv11 (You Only Look Once)	5
5.1. Modelos de 1 Etapa vs 2 Etapas	5
5.2. Non-Maximum Suppression (NMS)	5
6. Lógica Difusa: Las Matemáticas de la Sanción	5
6.1. El problema del "Serrucho"(Pérdida de Monotonidad en Mamdani)	6
6.2. La Solución: Takagi-Sugeno de Orden Cero	6
6.3. Las 5 Reglas Explícitas del Sistema de Sanción	6
7. Glosario Rápido de Conceptos de IA	6
8. Banco de Preguntas para la Defensa (Simulador)	7

1. Arquitectura del Sistema y Decisiones de Diseño

1.1. Pipeline de Procesamiento (Diagrama)

El sistema emplea una arquitectura **híbrida**. En lugar de usar una red neuronal *end-to-end* (que recibe la imagen y arroja el texto), el problema se divide en etapas especializadas. Esto garantiza mayor interpretabilidad, facilidad de depuración y determinismo.

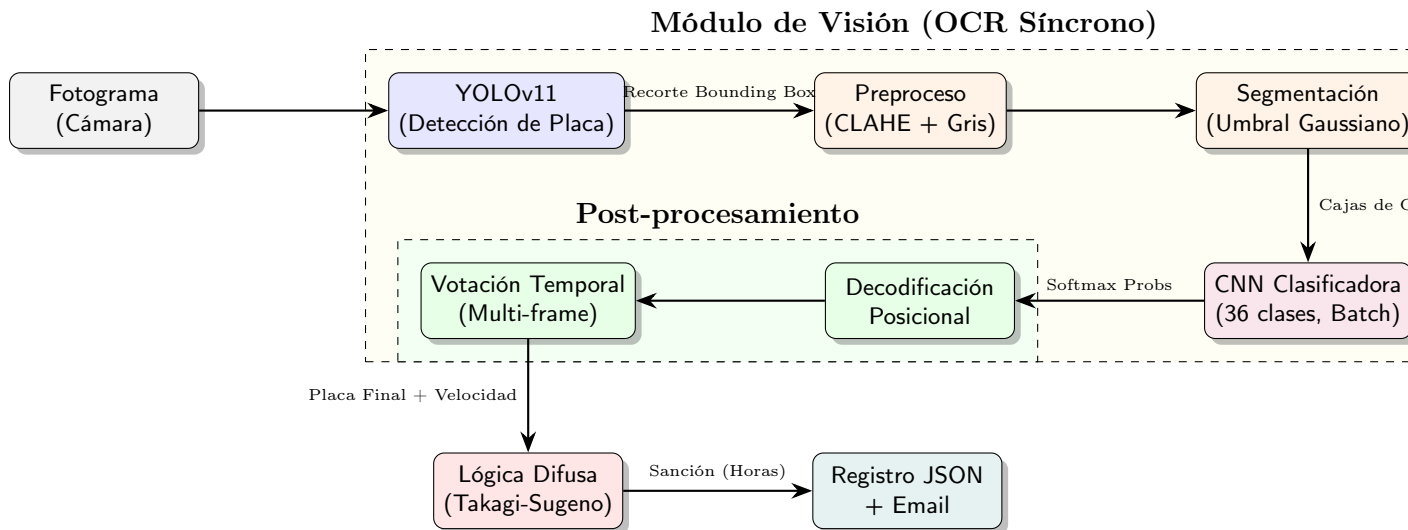


Figura 1: Arquitectura del Sistema de Radar y OCR.

1.2. El Debate Crítico: OCR Síncrono vs Asíncrono

Una de las decisiones arquitectónicas más fuertes fue **mantener el OCR en modo síncrono** (procesado en el bucle principal) y no enviar la tarea a un `ThreadPoolExecutor` asíncrono.

- **El problema asíncrono:** En un sistema con una sola GPU, el tracker de objetos (YOLO con ByteTrack) y el clasificador CNN compiten por los recursos CUDA. Si se usan hilos (asíncrono), la alternancia de contexto en la GPU genera cuellos de botella no deterministas (latencia espasmódica) y pérdida de fotogramas (lag).
- **La solución síncrona:** Se procesa exactamente **1 OCR por frame**. Para no desperdiciar recursos, las detecciones de YOLO se *ordenan por área*. Así, el "presupuesto" de 1 OCR se gasta siempre en el vehículo **más cercano y grande**, garantizando la máxima calidad de resolución de la placa sin bloquear la GPU para el tracking del resto de autos.

2. Visión por Computador Avanzada

2.1. Preprocesamiento y Resoluciones: El Umbral de 48px

El sistema extrae el *bounding box* de la placa directamente del fotograma original en alta resolución, no del tensor reescalado (416×416) de YOLO.

Para el OCR, el tamaño importa. Si la placa mide menos de **48 píxeles de altura**, se aplica un **escalado bicúbico** (`cv2.INTER_CUBIC`).

- **¿Por qué 48px?** Se bajó el umbral histórico de 72px a 48px empíricamente. Escalar placas medianas (50-70px) añadía ruido de interpolación innecesario.

- **¿Por qué Bicúbico y no Vecino Más Próximo?** El vecino más próximo genera artefactos de "escalera" (aliasing) que destruyen las curvas de las letras (O, C, 8). La bicúbica usa una fórmula polinomial sobre una vecindad 4×4 , preservando gradientes suaves en los bordes para la posterior binarización.

2.2. Binarización: Umbral Adaptativo Gaussiano vs Otsu

Otsu Global calcula un umbral único T minimizando la varianza intra-clase $\sigma_w^2(T)$ para toda la imagen. Falla estrepitosamente en placas vehiculares porque la iluminación nunca es uniforme (sombras de parachoques, reflejos del sol). Otsu terminaba oscureciendo media placa y fusionando caracteres.

El sistema usa **Umbral Adaptativo Gaussiano**. Matemáticamente, el umbral $T(x, y)$ para el píxel en la posición (x, y) se calcula de forma dinámica como una convolución cruzada con un kernel Gaussiano bidimensional $G(i, j)$ sobre una vecindad de tamaño $N \times N$, menos una constante C :

$$T(x, y) = \left(\sum_{i=-k}^k \sum_{j=-k}^k I(x+i, y+j) \cdot G(i, j) \right) - C \quad (1)$$

Intuición Didáctica: En lugar de usar una vara de medir estática para toda la imagen, cada píxel mira a sus vecinos. Si el barrio del píxel está en sombra (valores bajos), el promedio ponderado baja, así que el umbral para ser considerado "blanco" se vuelve más indulgente. Esto aísla perfectamente las letras del gradiente de luz.

2.3. Decodificación Posicional Categórica

La red CNN arroja probabilidades *Softmax* sobre 36 clases ($0-9, A-Z$). Existen letras y números visualmente indiscernibles a baja resolución ($O \leftrightarrow 0, I \leftrightarrow 1, Z \leftrightarrow 2, B \leftrightarrow 8$).

Dado que la placa ecuatoriana tiene el patrón estricto ABC-NNNN (3 letras, 3-4 dígitos), aplicamos un **argmax segmentado**:

$$c_i = \begin{cases} \arg \max_{k \in \mathcal{L}} p_k^{(i)} & \text{si } i < 3 \quad (\mathcal{L} = \text{Letras}) \\ \arg \max_{k \in \mathcal{D}} p_k^{(i)} & \text{si } i \geq 3 \quad (\mathcal{D} = \text{Dígitos}) \end{cases}$$

Si la CNN arroja $p(0) = 0.8, p(O) = 0.2$ en la posición $i = 1$, el sistema fuerza la decodificación a O , corrigiendo el error intrínseco del modelo.

3. Fundamentos Teóricos de Machine Learning y Deep Learning

3.1. Machine Learning Clásico vs Deep Learning

En el **Machine Learning clásico** (como SVMs o Random Forests), los ingenieros debían aplicar *Feature Engineering* manual: extraer métricas de la imagen como gradientes HOG o SIFT y pasarlas al algoritmo. El **Deep Learning** revolucionó esto usando redes neuronales profundas que aprenden la extracción de características y la clasificación de manera simultánea y automática (*End-to-End learning de representaciones*). La red aprende qué es lo más útil mirar.

3.2. Conceptos Nucleares y Matemáticas del Entrenamiento

- **Backpropagation y la Regla de la Cadena:** El entrenamiento es un problema de optimización. Para saber cómo corregir un peso w_{ij} en una capa profunda, usamos cálculo

diferencial. La Regla de la Cadena nos dice cuánto "culpa" tiene ese peso sobre el error final L :

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_{ij}} \quad (2)$$

Intuición: Calculamos la pendiente de la colina del error, y actualizamos el peso dando un paso hacia abajo: $w_{nuevo} = w_{viejo} - \alpha \frac{\partial L}{\partial w}$, donde α es la Tasa de Aprendizaje.

- **Funciones de Activación (ReLU vs Sigmoid):** La *Sigmoid* ($\sigma(x) = \frac{1}{1+e^{-x}}$) tiene una derivada máxima de 0.25. Si tienes una red de 10 capas, por la Regla de la Cadena multiplicas $0.25^{10} \approx 0.0000009$. **Resultado:** El gradiente "desaparece" (*Vanishing Gradient*) y las primeras capas nunca aprenden.

Por eso usamos **ReLU** ($f(x) = \max(0, x)$). Su derivada es exactamente 1 si $x > 0$. Multiplicar $1 \times 1 \times \dots = 1$, el gradiente fluye intacto como una autopista hasta la primera capa.

- **Optimizador AdamW vs Adam:** Adam clásico mete el decaimiento de pesos (*Weight Decay*) λ dentro de la fórmula del momentum, arruinando la regularización. **AdamW** lo desacopla, restando la penalización directamente del peso en el último paso:

$$\theta_t = \theta_{t-1} - \eta \left(\alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \right) - \eta \lambda \theta_{t-1} \quad (3)$$

Intuición: Esto obliga constantemente a los pesos θ a encogerse (hacia cero), lo que significa redes más simples que no memorizan los datos (*evita el Overfitting*).

- **Dropout:** Se apagan aleatoriamente (ej. 50 %) algunas neuronas multiplicando sus salidas por un filtro de Bernoulli. Matemáticamente, entrena un "ensamble" masivo de sub-redes compartiendo pesos.

4. Anatomía Profunda de una CNN (Convolutional Neural Network)

4.1. Convoluciones vs Redes Densas

Si una imagen tiene $48 \times 48 = 2304$ píxeles, una primera capa densa oculta con 1000 neuronas requeriría 2.3 Millones de pesos. Además, destruiría la estructura topológica 2D de la imagen (un píxel vecino al otro pierde significado).

Las **Convoluciones** solucionan esto. Matemáticamente, el valor del mapa de características S en la posición (i, j) al aplicar un filtro (kernel) K sobre la imagen I es:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n) \quad (4)$$

Intuición Didáctica: Imagina un filtro de Sobel para detectar líneas verticales: $K = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$.

Si deslizamos este filtro sobre una zona lisa (píxeles todos valen 100), la operación da $(-100 + 100) + (-200 + 200) \dots = 0$ (no hay borde). Pero si lo deslizamos sobre un borde brillante a la derecha (izquierda=0, derecha=200), la salida "explota" a números altos, detectando el borde. La CNN simplemente aprende los valores de estos filtros automáticamente en lugar de que nosotros los programemos. Esto garantiza **Parámetros Compartidos** (pocos pesos) e **Invarianza Espacial** (el borde se detecta igual arriba que abajo).

4.2. Jerarquía de Características

A medida que la red profundiza:

- **Capas Iniciales:** Aprenden filtros de Gabor (bordes, líneas, gradientes).
- **Capas Medias:** Combinan líneas para detectar texturas o formas básicas (esquinas, curvas de una ζ).
- **Capas Profundas:** Detectan partes de objetos complejos (el bucle de un "8", el cruce de una "X").

4.3. MaxPooling y Arquitectura ResNet

El **MaxPooling** (2×2) reduce la imagen a la mitad, seleccionando el píxel de mayor activación en un vecindario. Esto otorga invarianza a pequeñas traslaciones.

El clasificador **CNNPlacas** usa bloques **Residuales (ResNet)**. En redes estándar, la información se degrada al pasar por muchas capas. ResNet introduce una **Conexión de Salto** (*Skip Connection*): la entrada x se suma directamente a la salida de la capa $\mathcal{F}(x)$, es decir $H(x) = \mathcal{F}(x) + x$. Esto permite a la red aprender *residuos* y garantiza que el gradiente fluya directamente hacia atrás durante el *Backpropagation*.

5. Profundización en YOLOv11 (You Only Look Once)

5.1. Modelos de 1 Etapa vs 2 Etapas

Modelos clásicos como **Faster R-CNN** son de dos etapas: primero proponen miles de regiones Candidatas luego clasifican cada región. Son lentos. **YOLO** es de 1 etapa. Trata la detección como un problema de *regresión* en un solo paso. Toma la imagen, la divide en una grilla $S \times S$ y de una sola vez la red predice Bounding Boxes, Confianza y Probabilidad de Clases. Es extremadamente rápido y apto para Tiempo Real.

5.2. Non-Maximum Suppression (NMS)

Como YOLO predice sobre una grilla, a menudo arroja 5 cajas superpuestas para el mismo vehículo. **NMS** soluciona esto mediante la métrica matemática de **Intersección sobre Unión (IoU)**:

$$\text{IoU}(A, B) = \frac{\text{Área de Intersección}}{\text{Área de Unión}} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|} \quad (5)$$

Intuición del Algoritmo: 1. Ordena las cajas por Confianza y toma la mejor caja (Ej. 99 %). 2. Compara esta caja ganadora con todas las demás. Si la fórmula del IoU da mayor a 0.45, significa que las cajas se solapan demasiado (casi seguro son el mismo auto). 3. Elimina las cajas redundantes y repite el proceso para el siguiente objeto detectado.

6. Lógica Difusa: Las Matemáticas de la Sanción

Este proyecto implementa una arquitectura difusa *híbrida* (Mamdani para categorización y Takagi-Sugeno para sanción continua). Es crucial entender por qué se abandonó un sistema Mamdani puro para la sanción en horas.

6.1. El problema del "Serrucho" (Pérdida de Monotonidad en Mamdani)

En un inicio, la salida "horas de indisponibilidad" se modeló con Mamdani (defuzzificación por centroide de áreas). Si tenemos reglas consecutivas (Multa Leve $\rightarrow$ baja, Multa Moderada $\rightarrow$ media), y calculamos el **centroide de la unión de áreas recortadas**, ocurren caídas abruptas locales (el efecto "serrucho"). Cuando la velocidad aumenta y empieza a activar la siguiente regla, el área de la nueva regla al principio es pequeña, "jalando" el centroide hacia la zona compartida, lo que matemáticamente produce que a **mayor velocidad, haya ligeramente menor sanción localmente**. Esto viola el principio legal de proporcionalidad.

6.2. La Solución: Takagi-Sugeno de Orden Cero

El modelo de Sugeno descarta las áreas de salida. El consecuente de la regla i es un valor constante (un *singleton*) c_i . La salida total $H(v)$ es el promedio de los singletons ponderado por el grado de activación $\mu_i(v)$ de cada regla:

$$H(v) = \frac{\sum_{i=1}^n \mu_i(v) \cdot c_i}{\sum_{i=1}^n \mu_i(v)}$$

Dado que los singletons son estrictamente crecientes ($c_1 < c_2 < \dots < c_n$) y los antecedentes teselan el espacio (triángulos solapados 50 %), la curva resultante $H(v)$ es **estrictamente monótona y suave**. A más velocidad, **siempre** hay más sanción.

6.3. Las 5 Reglas Explícitas del Sistema de Sanción

Solo cuando el sistema Mamdani de nivel superior detecta la categoría **Multa** (> 20 km/h), se aplica la evaluación Sugeno de horas:

```
_REGLAS_SUGENO = [  
  # (nombre, [params MF_entrada], singleton_salida_horas)  
  ("multa_leve", [20, 24, 28], 5), # Exceso 1-8 km/h -> 5h  
  ("multa_moderada", [24, 28, 32], 18), # Exceso 5-12 km/h -> 18h  
  ("multa_alta", [28, 32, 36], 34), # Exceso 9-16 km/h -> 34h  
  ("multa_grave", [32, 36, 40], 52), # Exceso 13-20 km/h -> 52h  
  ("multa_extrema", [36, 40, 40, 40], 70), # Exceso >=16 km/h -> 70h  
]
```

7. Glosario Rápido de Conceptos de IA

Epoch (Época)	Una pasada completa de <i>todos</i> los datos de entrenamiento a través de la red neuronal.
Batch Size	El número de ejemplos procesados antes de actualizar los pesos del modelo. Usamos 256 para estabilidad del gradiente.
Learning Rate (Tasa de aprendizaje)	El tamaño del paso de corrección de los pesos. Se usa un <i>planificador OneCycleLR</i> (empieza bajo, sube a un pico, y baja asintóticamente).
AMP (Automatic Mixed Precision)	Uso de formato FP16 (16 bits) en vez de FP32 (32 bits) en la GPU. Duplica la velocidad de entrenamiento y reduce el uso de VRAM a la mitad sin perder precisión algorítmica.
Label Smoothing	En vez de forzar a la red a predecir 100 % de confianza para la respuesta correcta (vector [0, 0, 1, 0]), se asigna un 95 % a la correcta y 5 % distribuido al resto. Esto evita sobreconfianza y <i>overfitting</i> .

8. Banco de Preguntas para la Defensa (Simulador)

Esta sección contiene el núcleo para la defensa. El profesor intentará buscar huecos en la lógica del estudiante.

Pregunta Trampa: ¿Por qué no entrenaron una red (CNN/YOLO) para estimar la velocidad directamente de la imagen?

Respuesta: "Porque estimar velocidad desde un sensor monocular (1 sola cámara) mediante deep learning puro es un problema sub-determinado y pronóstico a errores masivos." Un modelo CNN requeriría mapeo 3D a 2D (estimación de profundidad y matriz de calibración intrínseca de la cámara) que varía dependiendo de cuánto zoom se aplique o el tamaño del auto. Nuestro enfoque geométrico con dos **líneas virtuales de distancia conocida** se rige por Física Clásica ($v = \Delta d / \Delta t$) y garantiza determinismo absoluto sin requerir calibración estéreo ni lidar.

Pregunta: ¿Por qué desecharon Otsu en la segmentación de caracteres?

Respuesta: "El método Otsu asume que la imagen tiene un histograma bimodal estricto (fondo y texto bien separados). En un ambiente al aire libre, un extremo de la placa puede tener sol directo y el otro la sombra del maletero del auto. El umbral global de Otsu promedia esto y destruye el texto en la sombra o quema el texto en el sol. Usamos **Umbral Adaptativo Gaussiano** porque calcula el umbral píxel a píxel en función de su vecindad local (21×21 píxeles), mitigando por completo las gradientes de luz."

Pregunta Trampa: Si su lógica difusa es tan precisa para dar las horas de sanción, ¿por qué no usaron una función matemática simple, tipo $f(x) = (x - 20) \times 3$?

Respuesta: "Por la **capacidad de modelado experto** que ofrece la lógica difusa." Una función lineal o polinomial trata todas las infracciones con la misma sensibilidad. La lógica difusa nos permite modelar la **heurística humana y legal**: la transición de leve a moderado requiere un nivel de severidad distinto al de grave a extremo. Al definir conjuntos difusos traslapados, podemos establecer mesetas, rampas de transición suave y umbrales no-lineales basados en reglas lingüísticas (IF 'velocidad es alta' THEN 'sanción es moderada'), lo cual es imposible de codificar de forma interpretable con una simple fórmula $f(x)$ o con rígidos If-Else que generan saltos abruptos.

Pregunta: En su clasificador CNN usaron Softmax. ¿Cuál es la diferencia entre Softmax y Sigmoid y por qué no usaron Sigmoid?

Respuesta: "La Sigmoid comprime cualquier número al rango $[0, 1]$ de forma independiente, asumiendo que un input podría pertenecer a múltiples clases a la vez (*Multi-label classification*). El **Softmax** escala las salidas de modo que **la suma de todas las probabilidades de las 36 clases sea exactamente 1.0** (*Multi-class classification*). Como un carácter en la placa solo puede ser **una sola cosa** (es una 'A' o es una 'B', no ambas), Softmax obliga a las probabilidades a competir entre sí, dando una distribución de confianza real."

Pregunta Trampa: Si imprimo una placa falsa en un papel y camino frente a su cámara, ¿me multan?

Respuesta: "Sí y no. YOLO detectaría la placa porque su patrón visual existe (bordes, texto). Sin embargo, el **filtro de clases** de nuestro tracker asocia la detección al *objeto padre* (Auto, Moto, Camión, Bus en COCO). Si una persona lleva el cartón, YOLO clasifica a la persona como **person** (clase 0) y no como vehículo (clases 2,3,5,7). Como solo medimos cruce de líneas para vehículos vivos, el peatón con el cartón no dispararía el flujo de velocidad."

Pregunta: He visto que su modelo CNN fue entrenado con placas de la India. ¿Cómo es posible que funcione en placas Ecuatorianas?

Respuesta: ^Esta es una distinción clave de nuestra arquitectura. Si hubiéramos usado una red OCR *End-to-End* (que lee toda la imagen de corrido, como CRNN), el modelo habría aprendido a leer el formato y la fuente de la India, fallando en Ecuador. Nosotros dividimos el problema: primero extraemos los caracteres sueltos mediante algoritmos de visión (Contornos y Componentes Conexos) y nuestra CNN sólo clasifica recortes individuales de 48×48 . **El número '5' o la letra 'M' de una placa de la India es tipográficamente idéntico al de Ecuador.** El modelo de *Deep Learning* solo reconoce los glifos universales; la lógica del formato ecuatoriano (ABC-NNNN) la aplicamos nosotros en el software."

Pregunta Trampa: Me dicen que usan "Votación Temporal" porque el OCR falla. ¿Su red neuronal es mala?

Respuesta: "No, nuestra red tiene un **93.6 % de precisión por carácter**, lo cual es excelente considerando el ruido exterior. El problema es estocástico: si una placa tiene 7 caracteres, la probabilidad de leer la placa perfecta en un sólo *frame* es $0.936^7 \approx 62\%$. Sin embargo, el vehículo aparece en nuestro campo de visión durante unos 10-15 frames. El error causado por el ruido (*motion blur*, sol, una gota de lluvia) es **aleatorio**, mientras que el carácter correcto es **constante**. Al acumular las lecturas y aplicar la Moda Matemática por columna (Votación Temporal), el ruido aleatorio se cancela y la señal constante emerge, dándonos secuencias correctas con una precisión práctica cercana al 100 %."

Pregunta: Veo que usan $lr = 10^{-3}$ y AdamW. ¿Por qué AdamW en vez del más famoso Adam regular?

Respuesta: AdamW (Adam con Weight Decay acoplado) soluciona un error matemático en la implementación original de Adam en PyTorch. Adam aplica la regularización L2 (*weight decay*) dentro del cálculo de los promedios móviles del gradiente, lo que hace que la regularización pierda fuerza, causando sobreajuste. **AdamW separa el *weight decay* sumándolo directamente a los pesos después de la optimización de gradiente, logrando que el modelo generalice mejor frente a datos no vistos.**"

Pregunta Trampa: En su `pipeline.py` vi que asignan un 'presupuesto' de OCR = 1 por frame. ¿Qué pasa si pasan dos vehículos exactamente al mismo tiempo y cruzan la línea?

Respuesta: Ambos serán detectados y su velocidad será medida, ya que YOLO y el Tracker (ByteTrack) son algoritmos paralelos y calculan el *bounding box* de todos en cada frame. Respecto a la placa, el presupuesto de $OCR = 1$ hace que el sistema clasifique

la placa del auto que tenga el **área más grande en pantalla** (sort por área). Como los autos cruzan en milisegundos distintos, el auto más cercano consume el OCR durante unos frames, y una vez que pasa, el siguiente auto se vuelve el de mayor área y hereda el presupuesto. Como tenemos de 10 a 15 frames efectivos en la zona de cámara lenta por segundo, hay frames más que suficientes para atender a ambos de forma síncrona sin bloquear el sistema."

Pregunta: ¿Qué es el hiperparámetro 'IoU' en YOLO y por qué es crucial para su tracker?

Respuesta: IoU (Intersection over Union) es la medida de solapamiento entre dos Bounding Boxes (Área de Intersección dividida para el Área de Unión). El tracker ByteTrack lo usa para el *Data Association*: si en el frame T hay una caja de auto y en el frame $T+1$ hay otra, el algoritmo calcula su IoU espacial. Si es muy alto (las cajas se solapan), el tracker concluye que es exactamente **el mismo auto** moviéndose y le mantiene su ID único. Sin IoU, el tracker no sabría cómo unir las detecciones entre *frames*."

Pregunta Trampa: Si su modelo tiene 1.2 millones de parámetros y el dataset de entrenamiento tiene 50,000 imágenes, ¿no se aprendió el dataset de memoria (Overfitting)?

Respuesta: "No, porque aplicamos fuertes técnicas de **Regularización**. Si bien los parámetros superan a las imágenes, usamos **Data Augmentation** en tiempo real (rotaciones aleatorias, escalado, ruido de brillo), lo que hace que la red nunca vea exactamente la misma imagen dos veces. Además, usamos **Dropout** en la arquitectura, obligando a que la red no pueda depender de caminos neuronales específicos, y **Weight Decay** (AdamW) que penaliza pesos muy grandes. Esto se comprueba porque nuestra precisión en el set de Entrenamiento es similar a la del set de Validación/Test: si hubiera overfitting, tendríamos 99 % en entrenamiento y 70 % en Test, pero tenemos ~ 91 % en ambos."

Pregunta: ¿Qué pasaría si paso una imagen de la placa completamente Negra (píxeles en 0) a su modelo CNN?

Respuesta: "La red haría los cálculos matemáticos normales (multiplicar 0 por los pesos), lo que resultaría en que el sumatorio de la primera capa oculta solo dependería de los **Bias** (sesgos) de las neuronas. Eventualmente, llegaremos a los *logits* finales. Al aplicar la función **Softmax**, se obtendría un vector de probabilidades que no tendrá sentido, pero cuya suma dará exactamente 1.0. Probablemente arrojará la clase con el sesgo más alto aprendido durante el entrenamiento, pero con una confianza (probabilidad) sumamente baja y repartida entre todas las clases, evidenciando que no hay un patrón."

Pregunta Trampa: Dicen que usan un 'Umbral Gaussiano'. Si la sombra oscurece la mitad de la placa, el Gaussiano bajará el umbral. ¿Qué impide que el ruido negro del asfalto en el fondo también se binarice como blanco?

Respuesta: .^{Esa} es una excelente observación sobre el Adaptive Thresholding. El umbral baja tanto en zonas oscuras que el ruido de fondo efectivamente puede binarizarse. Sin embargo, no nos importa, porque **solo aplicamos el umbral adaptativo sobre el Bounding Box apretado que ya recortó YOLO**. Como YOLO recorta estrictamente la chapa metálica de la placa, casi no hay asfalto en nuestro recorte. Además, la etapa

posterior de filtrado de contornos exige proporciones de Altura, Ancho y Área para considerar a un *blob* como un carácter válido. El ruido granulado que se genera se descarta en el filtrado por área mínima."